

UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida

Semestre 2 de 2026

Salvapantallas con OpenMP

Informe del Proyecto 1

Integrante	Carné
Pablo Cabrera	231156
Elizaar Canastuj	23384
Diego Ramírez	23782

Guatemala

5 de septiembre de 2026

Índice

Introducción y antecedentes	3
Diseño de la solución	4
Paralelización y método PCAM	5
Metodología experimental	6
Resultados y discusión	7
Conclusiones y recomendaciones	8
Referencias bibliográficas	9
Anexo 1 Diagrama de flujo general	10
Anexo 1 Actualización y sincronización	11
Anexo 2 Catálogo de funciones	12
Anexo 3 Bitácora de pruebas	16
Apéndice Evidencia y reproducción	19

Resumen

Se implementó en C un salvapantallas de N caminantes invisibles que dejan tres pares de huellas de colores. La solución mantiene una versión secuencial y dos variantes de OpenMP, con reparto estático y dinámico. Los cálculos de movimiento y colisiones utilizan dos arreglos de estados para evitar escrituras concurrentes sobre un mismo caminante.

La evaluación de CPU comprende 90 mediciones reales. Con $N = 1024$ y cuatro hilos, el speedup medio del cálculo fue 2.336 para static y 2.732 para dynamic. Con $N = 64$, ambas variantes paralelas resultaron más lentas. Estos resultados corresponden a Windows y a una compilación sin SDL2; no representan FPS de pantalla ni verifican el rendimiento gráfico en Linux.

La validación gráfica, las capturas de pantalla y el enlace del repositorio deben incorporarse antes de la entrega académica. La bitácora documenta las pruebas de CPU disponibles y sus límites, sin sustituirlas por datos simulados.

Introducción y antecedentes

El proyecto estudia cómo transformar un programa secuencial en una aplicación que aprovecha varios hilos en memoria compartida. El problema elegido es un salvapantallas de huellas: cada caminante tiene posición, velocidad y color propios; al desplazarse conserva únicamente sus tres pares de huellas más recientes. El trabajo repetido de actualización y detección de contactos permite comparar distintas formas de repartir el cálculo.

Objetivo general

Diseñar y evaluar una implementación secuencial y dos variantes paralelas con OpenMP del salvapantallas, comparando su tiempo de cálculo, speedup y eficiencia bajo cargas reproducibles.

Objetivos específicos

Aplicar PCAM al reparto de los caminantes; incorporar movimiento, rebotes e interacciones; proteger la actualización del estado compartido; parametrizar el programa; y registrar al menos diez mediciones por combinación de carga y versión.

Diseño de la solución

Representación de datos y parámetros

Walker almacena x , y , v_x , v_y , el acumulador temporal, tres canales de color y un arreglo de tres FootprintPair. Cada par contiene posición y ángulo. Simulation mantiene los punteros current y next. Config reúne los parámetros de entrada; Measurement registra tiempo de cálculo, tiempo total e hilos observados; Graphics contiene la ventana y el renderer de SDL2.

Parámetro	Valor predeterminado	Uso
--n	200	Cantidad de caminantes
--width y --height	800 y 600	Canvas mínimo admitido de 640×480
--speed y --radius	170 y 8	Rapidez máxima y radio de contacto
--foot-size y --foot-gap	10 y 14	Tamaño del pie y separación respecto al centro
--step-interval	0.12 s	Intervalo para renovar las huellas
--mode y --chunk	static y 32	Versión de cálculo y bloque dinámico
--seed y --fps	12345 y 60	Semilla y límite del modo interactivo

Movimiento y contacto entre caminantes

La inicialización calcula $v_x = v \cos(\theta)$ y $v_y = v \sin(\theta)$. En cada subpaso se actualiza la posición según $x_{\text{nuevo}} = x + v_x \Delta t$ e $y_{\text{nuevo}} = y + v_y \Delta t$. En una pared, se ajusta la posición al margen permitido y se invierte el signo de la componente normal de velocidad.

Dos discos se consideran en contacto si su distancia es menor que $2r$. La normal del contacto se obtiene de la diferencia de posiciones. Cuando la velocidad relativa apunta hacia el otro disco, se aplica un impulso normal de igual masa. También se corrige parcialmente la penetración. Para contactos simultáneos se promedian las contribuciones y se limita la rapidez: es una aproximación visual, no un simulador físico exacto.

La función advance divide el tiempo en subpasos para limitar el desplazamiento. Al cumplirse el intervalo de huellas, se desplaza el historial y se añade un nuevo par. Se dibujan primero las huellas antiguas, con menor intensidad. La complejidad es $O(N^2)$ por subpaso para contactos, $O(N)$ para dibujo y $O(N)$ para almacenamiento.

Paralelización y método PCAM

Partición

Cada tarea calcula el siguiente estado de un caminante i . Recorre los demás índices j para detectar contactos, calcula desplazamiento y rebotes, y renueva su historial. El bucle externo de N caminantes es la unidad de descomposición; el recorrido interno se mantiene secuencial.

Comunicación

Las tareas consultan el mismo arreglo `current`, que permanece de solo lectura durante el subpaso. El estado siguiente se escribe en `next[i]`. Así, un hilo no necesita leer resultados parciales de otro ni modificar directamente al vecino con el que colisiona. Cada pareja se analiza desde ambos extremos utilizando el mismo estado anterior.

Aglomeración y mapeo

La versión `seq` recorre los índices en un único hilo. La versión `static` distribuye las iteraciones con `schedule(static)`; `dynamic` solicita grupos de tamaño `chunk` mediante `schedule(dynamic, chunk)`. La opción `--threads` fija el número solicitado y se desactiva el ajuste dinámico del equipo. La medición registra además la cantidad de hilos observada en una región auxiliar.

Protección y sincronización

`default(none)` obliga a declarar las variables compartidas de cada región. Las variables locales de `update_one` pertenecen a cada llamada. La propiedad exclusiva de `next[i]` evita escrituras en competencia; la barrera implícita al finalizar `parallel for` asegura que todos los estados estén calculados antes de intercambiar los punteros `[1, 2]`. No se requiere `critical` ni un bloqueo global para actualizar cada caminante.

```
#pragma omp parallel for default(none) \
    shared(old,next,c,n,dt) schedule(static)
for (int i = 0; i < n; ++i)
    update_one(old, next, c, i, dt);
```

Costo y límites esperados

El reparto y la sincronización agregan costo a cada subpaso. Para cargas pequeñas, dicho costo puede superar el trabajo útil. Aun si el cálculo acelera, el dibujo secuencial y la presentación limitan la mejora del programa completo. Una región paralela persistente o una cuadrícula espacial son alternativas de optimización futura; no forman parte de esta versión.

Metodología experimental

Se utilizó la compilación de CPU de proyecto_final.c, con GCC 8.1.0 de MinGW y opciones -O2 -std=c11 -Wall -Wextra -Werror -fopenmp -DHEADLESS_ONLY. El entorno informa Windows NT 10.0.26200.0, arquitectura Intel64 Family 6 Model 158 Stepping 10 y ocho procesadores lógicos disponibles. No se identificó un modelo comercial de CPU ni se verificó la memoria RAM.

Factor controlado	Valor
Cargas N	64, 256 y 1024
Versiones	seq con 1 hilo; static y dynamic con 4 hilos
Repeticiones	10 por carga y versión; 90 mediciones en total
Duración de cada muestra	100 pasos medidos y 20 de calentamiento
Paso temporal y semilla	1/60 s y 12345
Canvas lógico y bloque dinámico	800 × 600 y chunk = 32
Gráficos y límite de FPS	Sin ventana; el límite interactivo no se aplica

Cada repetición reinicia el estado con la misma semilla. El calentamiento se excluye del tiempo registrado. El orden de las versiones rota entre repeticiones para reducir el sesgo por orden. No se controló de forma exclusiva la carga del sistema, la afinidad, la temperatura ni la frecuencia de CPU; los tiempos observados presentan variación y deben interpretarse como evidencia preliminar.

Definición de métricas

$\bar{T}$ = suma de diez tiempos / 10. El speedup de cálculo es $S = \bar{T}_{seq} / \bar{T}_{paralelo}$, usando compute_s. La eficiencia es $E = S / p$, donde p es la cantidad de hilos. Se informa como porcentaje multiplicando E por 100. El speedup total se calcula de la misma forma con total_s. No se promedian razones individuales.

En modo gráfico, el tiempo total incluye eventos, actualización, dibujo y presentación. En estas pruebas sin gráficos, $100 / \bar{T}_{total}$ mide pasos simulados por segundo. Aunque el CSV reutiliza el campo fps_or_cpu_steps_s, esos valores **no son FPS gráficos** ni demuestran que la aplicación sostenga 60 FPS. Windows utiliza QueryPerformanceCounter; Linux utiliza omp_get_wtime.

Resultados y discusión

N	Versión	Cálculo ms	S cálculo	E %	S total
64	seq	1.310	1.000	100.00	1.000
64	static	7.376	0.178	4.44	0.178
64	dynamic	6.956	0.188	4.71	0.189
256	seq	20.101	1.000	100.00	1.000
256	static	21.827	0.921	23.02	0.921
256	dynamic	18.609	1.080	27.00	1.080
1024	seq	501.355	1.000	100.00	1.000
1024	static	214.600	2.336	58.41	2.336
1024	dynamic	183.505	2.732	68.30	2.732

Tabla 1. Promedios de diez muestras de 100 pasos por configuración. Los tiempos corresponden al cálculo acumulado de toda la muestra, no a un solo cuadro. Fuente: archivos CSV generados por el programa.

Con $N = 64$, static obtiene $S = 0.178$ y dynamic $S = 0.188$. Ambos valores son menores que uno: ejecutar cuatro hilos empeora esta carga. Es consistente con un costo de coordinación mayor que el trabajo útil, aunque estas pruebas no aíslan cada fuente de sobre costo.

Con $N = 256$, los resultados medios se aproximan al punto de equilibrio: static alcanza 0.921 y dynamic 1.080. La dispersión observada impide atribuir una ventaja pequeña a la planificación sin ampliar la duración y controlar el entorno.

Con $N = 1024$, static reduce el tiempo medio de cálculo de 501.355 ms a 214.600 ms; dynamic lo reduce a 183.505 ms. Las eficiencias son 58.41% y 68.30%, respectivamente. La mayor carga permite amortizar mejor la coordinación. Estos resultados no implican que dynamic siempre sea superior.

El crecimiento cuadrático de la búsqueda de contactos incrementa el trabajo al aumentar N . La mejora observada se limita al mismo algoritmo y entorno; no permite concluir escalabilidad con más hilos ni comparar equipos. La autoevaluación del programa finalizó con SELF-TEST OK, comprobando equivalencia de estados y un contacto frontal aislado.

Conclusiones y recomendaciones

Conclusiones

1. La descomposición por caminante permite paralelizar el cálculo sin cambiar su orden interno de operaciones. Los dos arreglos de estado y la barrera implícita separan las lecturas del paso anterior de las escrituras del siguiente. La autoevaluación ejecutada encontró equivalencia entre seq, static y dynamic en los escenarios incluidos.
2. La mejora depende de la carga. Con 1024 caminantes y cuatro hilos se observaron speedups de cálculo de 2.336 y 2.732. Con 64 caminantes, las versiones paralelas fueron más lentas. Agregar hilos no garantiza acelerar una tarea pequeña.
3. Las mediciones sin SDL2 permiten evaluar el núcleo de simulación, pero no verifican el requisito de fluidez gráfica. La aceptación visual, los FPS en Linux y el comportamiento del renderer deben comprobarse mediante una ejecución con ventana.
4. El modelo de contactos y los subpasos son adecuados para estudiar una animación interactiva. La corrección aproximada de contactos múltiples y la búsqueda $O(N^2)$ limitan su fidelidad física y su crecimiento a cantidades muy grandes de caminantes.

Recomendaciones

Realizar la misma matriz de pruebas con SDL2 en el equipo de presentación, ampliando a uno, dos y cuatro hilos. Mantener semilla, cantidad de pasos, dimensiones y opciones de compilación; registrar CPU, RAM, sistema, compilador y renderer. Utilizar muestras de mayor duración y reportar dispersión además de promedios.

Capturar la ventana con N y FPS visibles, la salida de las mediciones y los resúmenes de cada carga. Determinar el mayor N que mantenga aproximadamente 60 FPS en ese equipo. Comparar siempre el tiempo completo con el tiempo de cálculo y conservar los CSV originales.

Completar el nombre del docente y el enlace del repositorio; adjuntar README y código fuente sin ejecutables. El historial de GitHub debe acreditar trabajo real desde al menos dos semanas antes de la entrega. Esa condición no puede acreditarse mediante un informe ni mediante resultados de rendimiento.

La estructura de este informe sigue los apartados enumerados en la rúbrica. Antes de entregar, contrastar tipografía, márgenes y normas de citas con la guía específica de informes UVG que utilice el curso.

Referencias bibliográficas

[1] OpenMP Architecture Review Board. (2021). OpenMP Application Programming Interface, versión 5.2. Sección 11.5, Worksharing-Loop Constructs.

<https://www.openmp.org/spec-html/5.2/openmpse66.html>

[2] OpenMP Architecture Review Board. (2021). OpenMP Application Programming Interface, versión 5.2. Sección 10.1, parallel Construct.

<https://www.openmp.org/spec-html/5.2/openmpse57.html>

[3] Foster, I. (1995). Designing and Building Parallel Programs. Sección 2.1, Methodical Design. Addison-Wesley. Versión electrónica alojada en Argonne National Laboratory.

<https://www.mcs.anl.gov/~itf/dbpp/text/node15.html>

[4] SDL Wiki. (s. f.). SDL2/SDL_RenderPresent. Documentación de la API SDL2.

https://wiki.libsdl.org/SDL2/SDL_RenderPresent

[5] GNU Project. (s. f.). GNU libgomp. Enabling OpenMP.

<https://gcc.gnu.org/onlinedocs/libgomp/Enabling-OpenMP.html>